

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчёт

Лабораторная работа 3

Выполнила:
Вдовина Анастасия
Группа J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Перенести сайт EventTix, собранный в первой и второй лабораторных работах, на Vue.js и превратить его в одностраничное приложение. Требования к результату: подключённый роутер, работа с внешним API через axios, разумное деление на компоненты и вынесение повторяющейся логики в composable.

Перенесены все семь страниц сайта и вся их функциональность, включая доработки из домашних работ — доступность, две темы оформления и SVG-спрайт.

Ход работы

Этап 1 — проект и слои

Проект создан командой `npm init vue@latest` на JavaScript, сборщик Vite, запуск на порту 8080. Данные по-прежнему отдаёт json-server с json-server-auth на порту 3000 — база и правила доступа из второй лабораторной работы перенесены без изменений.

Файлы разложены по слоям:

- **api** — экземпляр axios и классы запросов по сущностям (мероприятия, билеты, аккаунт);
- **stores** — хранилища Pinia: авторизация, мероприятия, билеты;
- **router** — маршруты и проверка доступа;
- **layouts** — общая обёртка страницы: шапка, подвал, слоты;
- **views** — семь страниц сайта;
- **components** — переиспользуемые части интерфейса;
- **composables** — логика, общая для разных страниц.

Компонент не обращается к серверу напрямую: страница берёт данные из хранилища, хранилище — из слоя api, и только он знает адреса запросов.

Чем SPA отличается от многостраничного сайта

В первых работах каждая ссылка вела к новому документу: браузер заново запрашивал HTML, заново выполнял скрипты и терял всё, что было в памяти — поэтому данные пользователя приходилось читать из `localStorage` на каждой странице. В SPA документ один, а роутер подменяет компонент внутри него. Состояние живёт в хранилище между переходами, и данные, загруженные один раз, остаются доступны.

```

src/
  api/          instance.js (axios и перехватчики), events.js,
                tickets.js, auth.js, index.js
  stores/       auth.js, events.js, tickets.js, index.js
  router/       index.js (маршруты и проверка доступа)
  composables/  useRequest.js, useAuthMessage.js, useTheme.js
  layouts/      BaseLayout.vue
  views/        HomePage, EventsPage, EventPage, LoginPage,
                RegisterPage, ProfilePage, OrganizerPage
  components/
    common/     HeaderComponent, ThemeToggle, IconComponent,
                ModalDialog, TicketTable
    events/     EventCard, EventFilters, ReviewList
  assets/       main.css
  App.vue, main.js

```

Этап 2 — маршруты и доступ

Семь страниц сайта стали семьёю маршрутами: главная, каталог, мероприятие с параметром `:id`, вход, регистрация, личный кабинет и кабинет организатора. Страницы подключены лениво — код страницы скачивается при первом переходе на неё, а не весь сразу.

Во второй лабораторной работе доступ проверялся на каждой странице отдельно (`requireAuth`, `requireRole`). Теперь проверка одна, в `beforeEach`: у маршрута в `meta` указано, нужен ли вход, нужна ли роль организатора и не следует ли, наоборот, увести уже вошедшего пользователя со страницы входа.

Навигационные хуки

`beforeEach` выполняется перед каждым переходом и решает, состоится ли он: вернуть `true` — разрешить, вернуть маршрут — перенаправить. Это по-прежнему клиентская проверка и не замена защите на сервере: она прячет страницу, а данные закрывают правила доступа `json-server-auth` и проверка токена.

Этап 3 — запросы через axios

Создан один экземпляр `axios` с базовым адресом сервера и двумя перехватчиками. Первый подставляет в каждый запрос заголовок с токеном, второй ловит ответ 401: токен живёт час, и когда он истёк, продолжать работу бессмысленно — сессия очищается, пользователь уходит на страницу входа.

Запросы собраны в классы по сущностям: мероприятия (список с фильтрами, одно мероприятие, создание, сохранение отзывов), билеты (покупка, смена статуса, возвраты), аккаунт (вход, регистрация, изменение данных). Фильтры каталога уходят на сервер `query`-параметрами, как и раньше: `q`, `type`, `city`, `date_gte`.

Зачем перехватчики

Без них заголовков с токеном пришлось бы добавлять в каждый запрос, а реакцию на истёкший токен — в каждый обработчик ошибки. Перехватчик описывает это один раз для всех запросов сразу и остаётся единственным местом, где вообще упоминается авторизация на уровне транспорта.

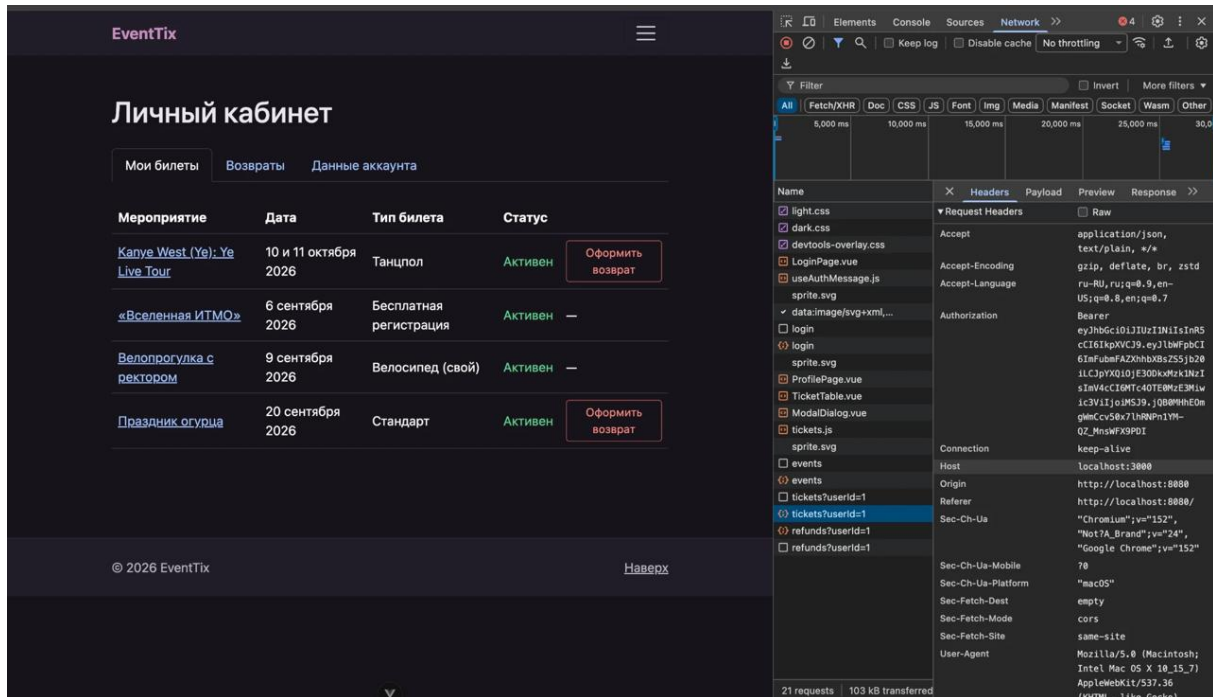


Рис. 1: Запрос к защищённому ресурсу: заголовок с токеном подставил перехватчик, в коде страницы его нет.

Этап 4 — деление на компоненты

Повторяющиеся части интерфейса вынесены в компоненты. Шапка с навигацией и переключателем темы, подвал и слоты для содержимого собраны в `BaseLayout` — каждая страница описывает только то, чем отличается от остальных.

- `EventCard` — карточка мероприятия. Получает мероприятие через `props`; уровень заголовка тоже приходит извне, потому что в каталоге карточки стоят под `h1`, а на главной — под заголовком раздела;
- `EventFilters` — форма фильтров, связана со страницей через `v-model` и сообщает о нажатии кнопок событиями;
- `ModalDialog` — диалоговое окно: заголовок приходит через `props`, содержимое и кнопка подтверждения — через слоты. Один компонент обслуживает покупку билета, оформление возврата и создание мероприятия;
- `TicketTable`, `ReviewList` — таблица билетов и список отзывов, оба сообщают о

действиях наружу событиями;

- `IconComponent`, `ThemeToggle` — иконка из спрайта и кнопка переключения темы.

Во второй лабораторной работе диалоговые окна показывал скрипт Bootstrap, а их содержимое собиралось строками прямо в JavaScript. Теперь разметка диалога описана один раз в шаблоне компонента, а открыт он или нет — обычное значение в данных страницы.

props, события и слоты

Данные идут в компонент через props — сверху вниз. Обратно компонент не меняет чужие данные, а сообщает о событии, и решение принимает родитель. Слот — это дырка в шаблоне, куда родитель кладёт свою разметку: благодаря ему один диалог подходит и для покупки билета, и для возврата.

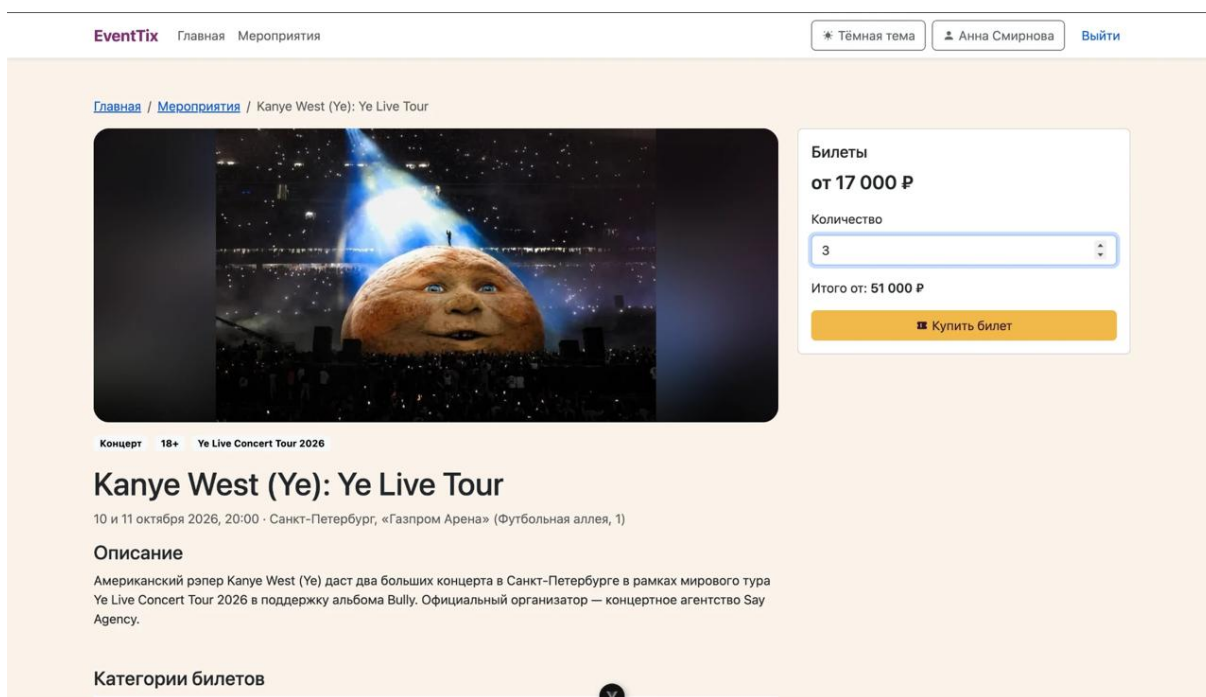


Рис. 2: Страница мероприятия: количество билетов и итоговая сумма связаны вычисляемым свойством, пересчёт происходит без обращения к DOM.

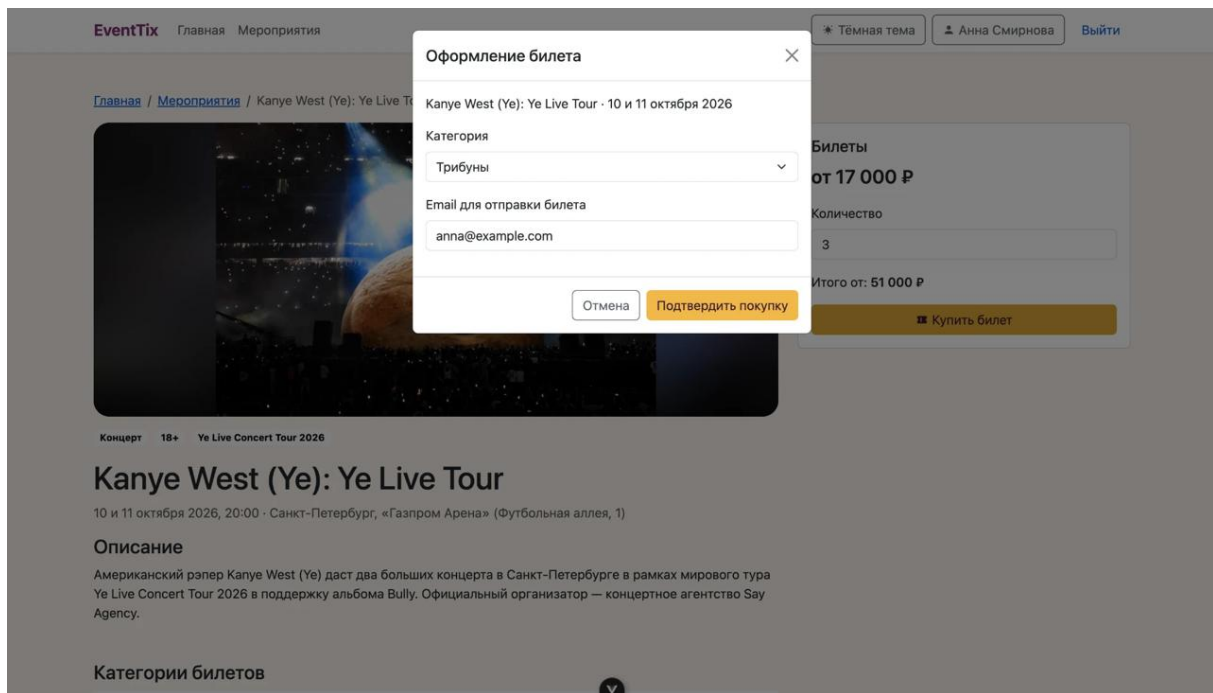


Рис. 3: Оформление билета в диалоговом окне — том же компоненте, что используется для возврата и для создания мероприятия.

Этап 5 — composables

В отдельные файлы вынесено то, что повторялось на разных страницах и не относится к хранению данных:

- **useRequest** — три состояния запроса. Принимает функцию загрузки и отдаёт признак загрузки, текст ошибки и запуск. Используется на четырёх страницах, поэтому «загружаем...», «не удалось загрузить» и сама отрисовка больше нигде не дублируются;
- **useAuthMessage** — перевод ответов сервера на русский: `Incorrect password` превращается в «Неверный email или пароль». Нужен на входе и на регистрации;
- **useTheme** — переключение темы и запоминание выбора, перенесено из третьей домашней работы.

Composable или хранилище

Хранилище держит данные, общие для всего приложения: кто вошёл, какие мероприятия загружены. Composable не хранит общих данных — он описывает поведение и при каждом вызове создаёт свой набор значений. Поэтому две страницы с **useRequest** не мешают друг другу: у каждой свои признак загрузки и ошибка.

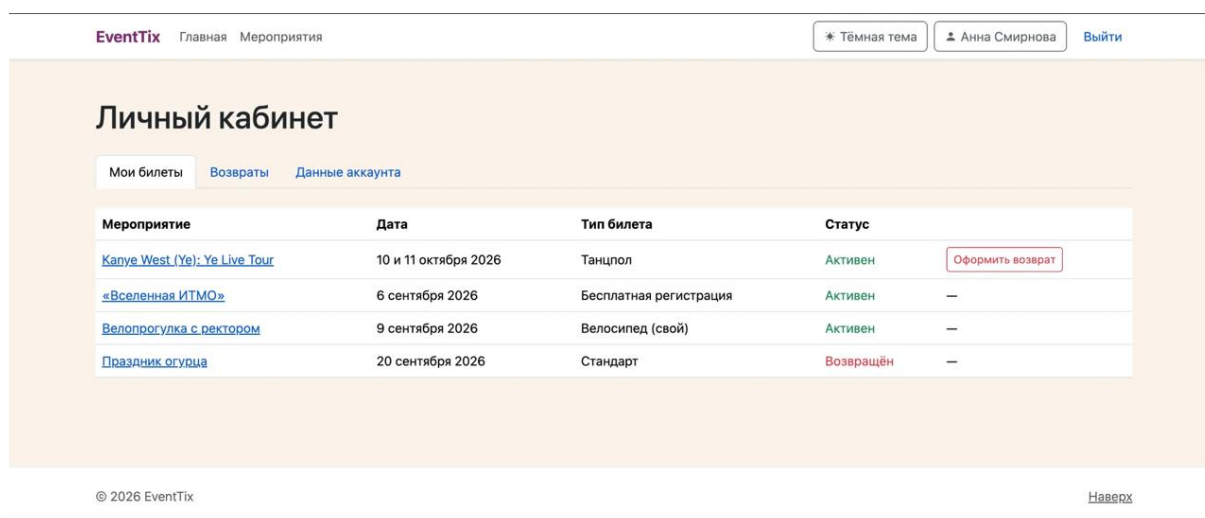


Рис. 4: Личный кабинет: вкладки, билеты и история возвратов. Возврат оформляется сменой статуса билета, а не удалением записи.

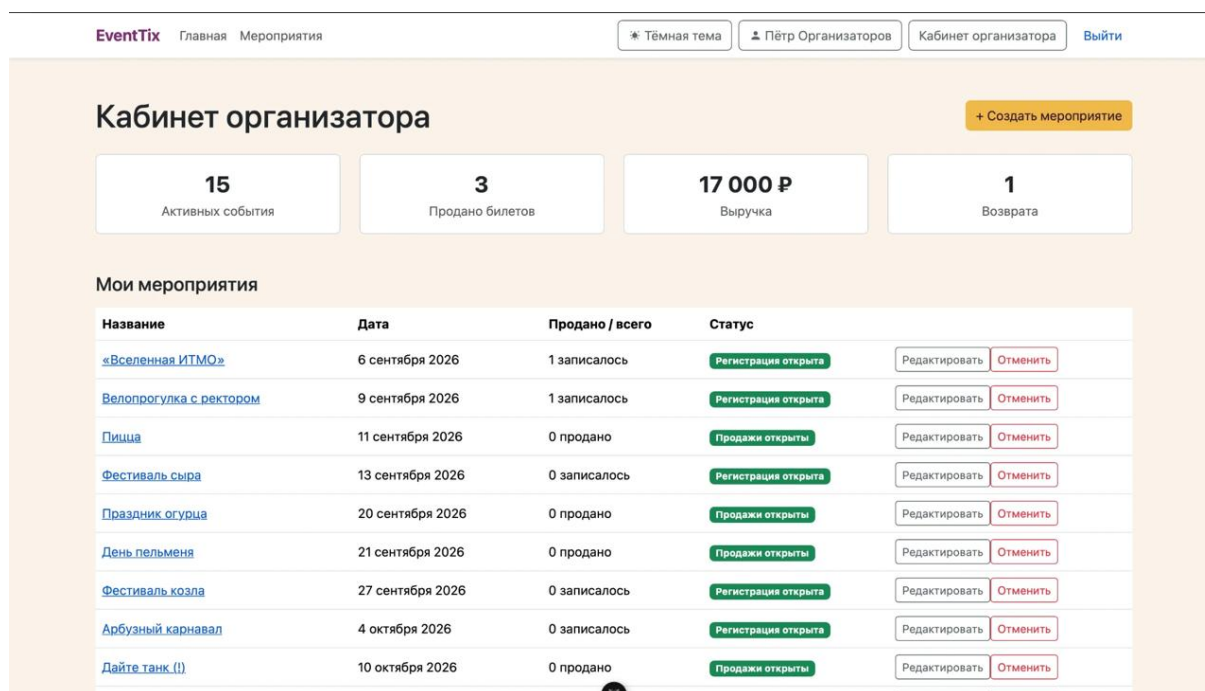


Рис. 5: Кабинет организатора: показатели продаж посчитаны вычисляемыми свойствами из загруженных билетов и возвратов.

Этап 6 — перенос доработок из домашних работ

Вместе со страницами перенесены доработки, сделанные в домашних работах.

- **Доступность.** Ссылка перехода к основному содержимому, подписи у навигации и форм, `role="search"` у поиска, живые области для списков, скрытые подписи таблиц, связь подсказки с полем пароля. Меню на узком экране сворачивается, и состояние кнопки отражено в `aria-expanded`. Проверка Lighthouse — 100 баллов,

как и у статической версии;

- **Темы.** Файлы светлой и тёмной темы подключены ссылками с атрибутом `media`, кнопка в шапке переключает их и запоминает выбор. Bootstrap подключён ссылкой перед файлами тем — иначе его переменные оказались бы в каскаде последними и перекрыли тёмную тему;
- **Спрайт.** Иконки остались в одном файле и подключаются ссылкой на `id`. Разметка `<use>` описана один раз в `IconComponent`, а в местах использования осталось имя иконки.

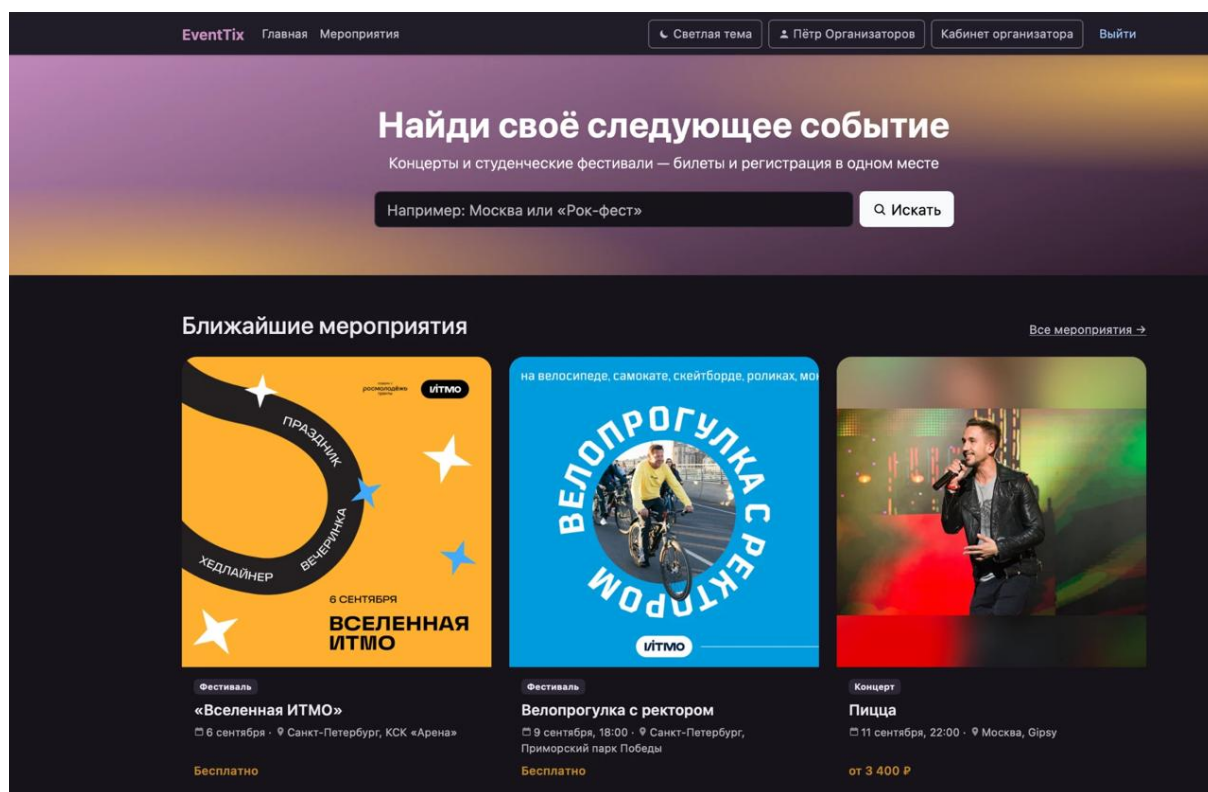


Рис. 6: Тёмная тема в собранном приложении: разметка общая, меняются только значения переменных.

Вывод

Сайт перенесён на Vue.js без потери функциональности: те же семь страниц, те же запросы к API и те же доработки по доступности, темам и иконкам. Переходы между страницами выполняет роутер, он же проверяет доступ; запросы собраны в слое `api` с перехватчиками токена; интерфейс разбит на компоненты, а повторяющаяся логика вынесена в `composables`. Главное отличие от прошлых работ — интерфейс больше не обновляется вручную: страница описывает, как данные должны выглядеть, а остальное делает фреймворк.